Architecture

This system is built around a two-process design; an API server and a worker; that talk to each other through Redis, not directly. The API server handles HTTP requests and the worker does the heavy lifting of processing CSV files. They don't know about each other. They just share a Redis queue and a MongoDB database.

Why two processes? Because uploads and processing have completely different resource profiles. The API server is I/O-bound; it's just routing requests, generating presigned URLs, and pushing jobs onto a queue. The worker is CPU and memory-bound; it's parsing CSVs, validating rows, and detecting duplicates. Keeping them separate means, you can scale each one independently. If you're getting a lot of uploads but processing is fine, add more API replicas. If there's a backlog of jobs, spin up more workers. BullMQ handles job distribution automatically.

The upload flow is intentionally indirect. The client never sends the file to our API server. Instead, it asks for a presigned S3 URL, uploads directly to S3, and then tells our server "Hey, the file is there now, go process it." This means the API server never touches file bytes; it stays lightweight and can handle thousands of concurrent uploads without breaking a sweat. S3 handles heavy transfers.

For processing, the worker streams CSVs from S3 row by row using Node.js streams. It never loads the entire file into memory. A 2GB file uses roughly the same memory as a 2MB file. Each row goes through validation (checking required fields, data types, custom regex patterns) and duplicate detection.

Duplicate detection uses a hybrid approach. For smaller files (under 50MB), it uses SHA-256 hashes stored in a Set; exact and fast. For larger files, it switches to a Bloom filter, which uses way less memory but has roughly a 1% false positive rate. The threshold is configurable. This was a practical tradeoff, you could use hashes for everything, but on a 500MB file with millions of rows, that hash set would eat up significant memory.

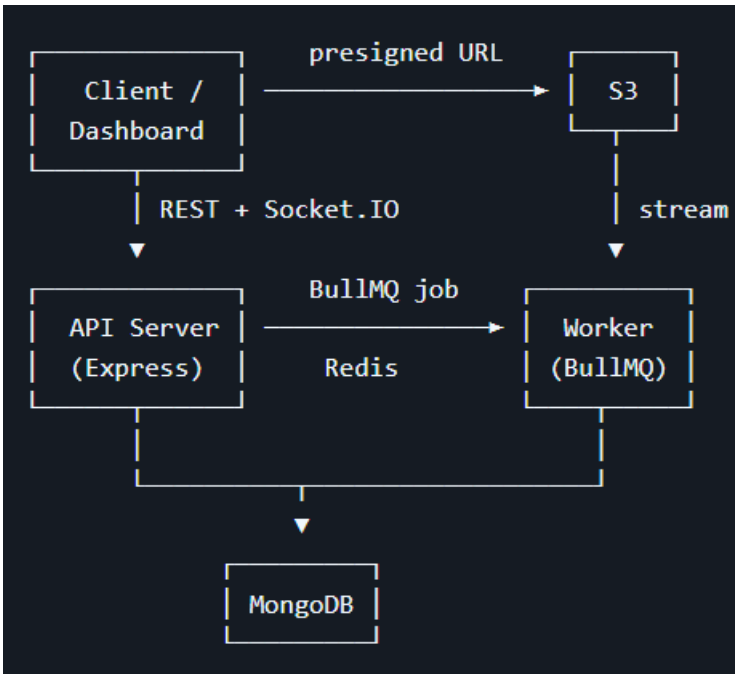
Real-time updates go through Socket.IO with a Redis adapter. The tricky part is that the worker doesn't run a Socket.IO server, it uses @socket.io/redis-emitter to push events into Redis pub/sub, and the API server's Socket.IO instance picks them up and forwards them to connected clients.

This way the worker stays focused on processing, and the API server stays the single point of contact for clients.

Data access uses the repository pattern. All MongoDB operations go through interfaces (IJobRepository, IFileRepository, IResultRepository), and the actual

Mongoose implementations sit behind them. If you ever wanted to swap MongoDB for Postgres or DynamoDB, you'd write new repository classes without touching the service layer. It's a bit more code upfront, but it keeps things testable and flexible.

Caching is straightforward, Redis with TTL. Stats get cached for short periods (10 seconds for stats) so the dashboard isn't hammering MongoDB on every poll. The worker invalidates the cache when jobs complete.



Design Decisions

1) Presigned URLs over multipart upload through the server; This was the biggest architectural choice. Having the server proxy file uploads is the simpler approach, but it doesn't scale. Every upload would tie up a connection and memory on the API server. With presigned URLs, S3 handles all the transfer complexity, and our server just manages the metadata. The tradeoff is a slightly more complex client flow (two API calls instead of one), but it's worth it.

2) BullMQ over simpler alternative; We needed priority support, retry logic, rate limiting, and concurrency control. BullMQ gives all of that out of the box with Redis as the backend. We were already using Redis for Socket.IO and caching, so it didn't add infrastructure complexity.

3) Streaming over batch processing; Loading an entire CSV into memory, parsing it, and then processing would be simpler code. But it puts a hard ceiling on file size based on available

memory. Streaming removes that ceiling. The code is a bit more involved with backpressure handling, but it means the system works the same whether you throw a 10KB file or a 10GB file at it.

4) Bloom filter for large file deduplication; A pure hash-based approach is more accurate, but memory usage grows linearly with row count. At millions of rows, that becomes a real problem for a worker that might be processing multiple files concurrently. The Bloom filter keeps memory bounded at the cost of a small false positive rate. For most use cases, flagging 1% of unique rows as duplicates is acceptable; and it's configurable if it's not.

5) Repository pattern over direct Mongoose calls; The services never import Mongoose directly. This makes unit testing straightforward (you can mock the repository interface) and keeps the door open for database changes. It's more indirection than a small project needs, but for something designed to scale, it pays off.

6) Redis for three things (queue, cache, pub/sub); Instead of running separate infrastructure for each concern, Redis handles all three. Less operational overhead, fewer things to monitor, and they all benefit from Redis being fast. The tradeoff is that a Redis outage affects everything, but in practice you'd run a managed Redis cluster in production anyway.

Assumptions and Limitations

1) CSV only; Right now the system only handles CSV files. The processor is built around csv-parser and the validation logic assumes tabular row data. You could extend it to handle Excel, JSON, or other formats by writing new processors.

2) MongoDB is external; The Docker Compose setup includes Redis, the API, the worker, and the dashboard, but not MongoDB. This was intentional; running MongoDB in a single Docker container isn't a good production setup anyway.

3) No authentication; There's no auth layer. The API is open. Adding auth directly would be straightforward (middleware on the routes), but it wasn't part of the scope.

4) Error details are capped at 100 per job; If a CSV has 50,000 invalid rows, the system only stores the first 100 error details. This prevents a single bad file from bloating the result document in MongoDB. The total counts (invalid rows, duplicate rows) are still accurate; you just don't get the line-by-line breakdown beyond 100.

5) Bloom filter false positives; For files over 50MB, the duplicate detector uses a Bloom filter with roughly a 1% false positive rate. That means about 1 in 100 unique rows might get incorrectly flagged as a duplicate. For most data processing tasks this is fine, but if you need exact deduplication on large files, you'd need to increase the hash threshold at the cost of memory.

6) S3 orphan cleanup relies on lifecycle rules; If a client gets a presigned URL, uploads to S3, but never calls the confirm endpoint, that file sits in S3 unclaimed. The system relies on S3 lifecycle rules (configured in infra/) to auto-delete objects older than 1 day that were never confirmed. There's no background job sweeping for orphans.

7) Single queue; All jobs go through one BullMQ queue (file-processing). Priority is supported within that queue, but there's no separate queue for different file types or processing tiers. For most workloads this is fine, but if you had vastly different SLA requirements for different job types, you'd want multiple queues.